Ejercicio Práctico — Regresión

Tarea 1: GridSearchCV aplicado a Gradient Boosting y RNA

Se adapta el ejemplo de optimización de hiperparámetros (originalmente para Random Forest) para aplicarlo a Gradient Boosting y a la Red Neuronal (MLP).

1.1 — Gradient Boosting con GridSearchCV

Los hiperparámetros más relevantes para GradientBoostingRegressor son el número de estimadores, la tasa de aprendizaje y la profundidad máxima de cada árbol base.

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV

# 1. Modelo base
gb = GradientBoostingRegressor(random_state=42)

# 2. Grilla de hiperparámetros
param_grid_gb = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth':     [3, 5, 7]
}

# 3. Búsqueda con validación cruzada (5 folds)
grid_gb = GridSearchCV(
    estimator=gb,
    param_grid=param_grid_gb,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)

# 4. Ajuste
grid_gb.fit(X_train, y_train)

# 5. Resultados
print('Mejores parámetros (GB):', grid_gb.best_params_)
print(f'Mejor R² en CV:          {grid_gb.best_score_:.4f}')

# 6. Evaluación en test
best_gb = grid_gb.best_estimator_
preds_gb = best_gb.predict(X_test)
print(f'R² en test:              {r2_score(y_test, preds_gb):.4f}')
print(f'MSE en test:             {mean_squared_error(y_test, preds_gb):.4f}')
```

Consejo: learning_rate y n_estimators tienen una relación inversa: tasas bajas requieren más árboles para converger, pero generalizan mejor. Se recomienda probar valores entre 0.01 y 0.2.

1.2 — Red Neuronal (MLP) con GridSearchCV

Para MLPRegressor los hiperparámetros clave son la arquitectura de capas ocultas, la función de activación y la tasa de aprendizaje inicial.

```
from sklearn.neural_network import MLPRegressor

# 1. Modelo base  (max_iter alto para asegurar convergencia)
mlp = MLPRegressor(max_iter=2000, random_state=42, early_stopping=True)

# 2. Grilla de hiperparámetros
param_grid_mlp = {
    'hidden_layer_sizes': [(50,), (100,), (100, 50), (100, 100, 50)],
    'activation':         ['relu', 'tanh'],
    'learning_rate_init': [0.001, 0.01, 0.1]
}

# 3. Grid Search
grid_mlp = GridSearchCV(
    estimator=mlp,
    param_grid=param_grid_mlp,
    cv=5,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)

# 4. Ajuste
grid_mlp.fit(X_train, y_train)

# 5. Resultados
print('Mejores parámetros (MLP):', grid_mlp.best_params_)
print(f'Mejor R² en CV:           {grid_mlp.best_score_:.4f}')

# 6. Evaluación en test
best_mlp = grid_mlp.best_estimator_
preds_mlp = best_mlp.predict(X_test)
print(f'R² en test:               {r2_score(y_test, preds_mlp):.4f}')
print(f'MSE en test:              {mean_squared_error(y_test, preds_mlp):.4f}')
```

Nota sobre tiempo de cómputo: Con una grilla de 4×2×3 = 24 combinaciones y cv=5, se entrenan 120 modelos. Para RNA esto puede ser lento. Alternativa: usar RandomizedSearchCV con n_iter=20 para explorar el espacio de forma más eficiente.

1.3 — Comparación final de todos los modelos optimizados

```
results_opt = pd.DataFrame([
    {'Modelo': 'Random Forest (opt)',      'R2': r2_score(y_test,
grid_search.best_estimator_.predict(X_test))},
    {'Modelo': 'Gradient Boosting (opt)',  'R2': r2_score(y_test, preds_gb)},
    {'Modelo': 'Red Neuronal MLP (opt)',   'R2': r2_score(y_test, preds_mlp)},
])
print(results_opt.sort_values('R2', ascending=False).to_string(index=False)
```

Tarea 2: Algoritmos de Gradient Boosting — XGBoost, LightGBM y CatBoost

Los tres son variantes altamente optimizadas del algoritmo de Gradient Boosting. Cada uno introduce mejoras en velocidad, memoria y manejo de datos para superar las limitaciones del GradientBoostingRegressor clásico de scikit-learn.

2.1 — XGBoost (Extreme Gradient Boosting)

Desarrollado por Tianqi Chen (2016). Fue el algoritmo dominante en competencias de Kaggle durante varios años. Introduce regularización L1/L2 directamente en la función objetivo, poda de árboles nivel a nivel (depth-wise) y soporte nativo para valores nulos.

Instalación y caso de aplicación: predicción de precios de vivienda

```
# pip install xgboost
import xgboost as xgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

# Datos reales de vivienda en California
data = fetch_california_housing()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Modelo XGBoost
model_xgb = xgb.XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,        # Fracción de muestras por árbol (reduce overfitting)
    colsample_bytree=0.8, # Fracción de features por árbol
    reg_alpha=0.1,        # Regularización L1
    reg_lambda=1.0,       # Regularización L2
    random_state=42,
    n_jobs=-1
)

model_xgb.fit(X_train, y_train,
              eval_set=[(X_test, y_test)],
              verbose=False)

preds = model_xgb.predict(X_test)
print(f'XGBoost  — R²: {r2_score(y_test, preds):.4f}  MSE:
{mean_squared_error(y_test, preds):.4f}')
```

2.2 — LightGBM (Light Gradient Boosting Machine)

Desarrollado por Microsoft (2017). Su principal innovación es el crecimiento de árboles leaf-wise (en lugar de depth-wise), lo que permite mayor precisión con menos iteraciones. Introduce GOSS (Gradient-based One-Side Sampling) y EFB (Exclusive Feature Bundling) para manejar millones de filas y miles de columnas de forma eficiente.

Instalación y caso de aplicación: predicción de demanda energética

```
# pip install lightgbm
import lightgbm as lgb
import numpy as np

# Simulamos datos de demanda eléctrica (hora, temperatura, día semana)
np.random.seed(0)
n = 10000
hora        = np.random.randint(0, 24, n)
temperatura = np.random.uniform(5, 40, n)
dia_semana  = np.random.randint(0, 7, n)
X = np.column_stack([hora, temperatura, dia_semana])
y = (200 + 15*temperatura - 5*(hora-12)**2
     + np.where(dia_semana >= 5, -30, 0)
     + np.random.normal(0, 10, n))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Modelo LightGBM
model_lgb = lgb.LGBMRegressor(
    n_estimators=500,
    learning_rate=0.05,
    num_leaves=63,        # Controla complejidad (2^max_depth - 1 es típico)
    min_child_samples=20, # Mínimo muestras por hoja
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

model_lgb.fit(X_train, y_train,
              eval_set=[(X_test, y_test)],
              callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)])

preds = model_lgb.predict(X_test)
print(f'LightGBM — R²: {r2_score(y_test, preds):.4f}  MSE:
{mean_squared_error(y_test, preds):.4f}')
```

2.3 — CatBoost (Categorical Boosting)

Desarrollado por Yandex (2017). Su diferenciador clave es el manejo nativo de variables categóricas sin necesidad de codificarlas manualmente (label encoding u one-hot). Usa Ordered Boosting para evitar el target leakage durante el entrenamiento y es especialmente robusto con datasets pequeños.

Instalación y caso de aplicación: predicción de churn de clientes

```
# pip install catboost
from catboost import CatBoostRegressor
import pandas as pd
import numpy as np

# Datos simulados con variables categóricas (plan, región)
np.random.seed(42)
n = 2000
planes   = np.random.choice(['basico', 'estandar', 'premium'], n)
regiones = np.random.choice(['norte', 'sur', 'centro', 'oriente'], n)
edad     = np.random.randint(18, 70, n)
consumo  = np.random.uniform(0, 100, n)

# Target: meses hasta churn (problema de regresión)
y = (12 + 6*(planes == 'premium') + 3*(planes == 'estandar')
     - 0.1*edad + 0.05*consumo + np.random.normal(0, 2, n)).clip(1, 36)

df = pd.DataFrame({'plan': planes, 'region': regiones,
                   'edad': edad,   'consumo': consumo})

X_train, X_test, y_train, y_test = train_test_split(
    df, y, test_size=0.2, random_state=42)

# CatBoost detecta automáticamente las columnas categóricas
cat_features = ['plan', 'region']

model_cat = CatBoostRegressor(
    iterations=400,
    learning_rate=0.05,
    depth=6,
    cat_features=cat_features,
    random_seed=42,
    verbose=100
)

model_cat.fit(X_train, y_train, eval_set=(X_test, y_test))
preds = model_cat.predict(X_test)
print(f'CatBoost — R²: {r2_score(y_test, preds):.4f}  MSE:
{mean_squared_error(y_test, preds):.4f}')
```

2.4 — Tabla comparativa

Característica	XGBoost	LightGBM	CatBoost
Crecimiento de árbol	Depth-wise	Leaf-wise	Oblivious (simétrico)
Categoriales nativo	No	Parcial	Sí (sin preprocesar)
Velocidad entrenamiento	Media	Alta (más rápido)	Media-Alta
Uso de memoria	Medio	Bajo	Medio
Datasets pequeños	Bueno	Regular	Muy bueno
Regularización	L1 + L2	L1 + L2	Ordenada (target leak)
Caso de uso típico	Tabulares generales	Big data / alta dim.	Datos con categorías

Tarea 3: Regresión Ridge y Lasso

Ridge y Lasso son extensiones de la regresión lineal que incorporan un término de penalización (regularización) en la función de pérdida. Esto limita el tamaño de los coeficientes y resuelve dos problemas comunes: el sobreajuste (overfitting) y la multicolinealidad entre variables.

3.1 — Función de pérdida comparada

Regresión Lineal OLS: Minimiza $\Sigma(y_i - \hat{y}_i)^2$

Ridge (L2): Minimiza $\Sigma(y_i - \hat{y}_i)^2 + \alpha \cdot \Sigma\beta_j^2$ → penaliza el cuadrado de los coeficientes

Lasso (L1): Minimiza $\Sigma(y_i - \hat{y}_i)^2 + \alpha \cdot \Sigma|\beta_j|$ → penaliza el valor absoluto de los coeficientes

El parámetro α (alpha) controla la intensidad de la penalización: α = 0 equivale a regresión lineal sin regularización; valores altos de α reducen los coeficientes hacia cero.

3.2 — Diferencia clave: selección de variables

Aspecto	Ridge	Lasso
Norma de penalización	L2 (cuadrado)	L1 (valor absoluto)
Efecto sobre coeficientes	Los encoge pero nunca los elimina	Puede forzarlos exactamente a cero
Selección de variables	No (todos permanecen)	Sí (selección automática)
Multicolinealidad	Muy robusto	Elige una y descarta las correlacionadas
Solución analítica	Sí ($\beta = (X^TX + \alpha I)^{-1}X^Ty$)	No (optimización iterativa)
Cuándo usar	Todas las variables aportan	Se sospecha que muchas son irrelevantes

3.3 — Implementación con selección de alpha óptimo

```
from sklearn.linear_model import Ridge, Lasso, RidgeCV, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np
import matplotlib.pyplot as plt

# Escalar es OBLIGATORIO antes de Ridge/Lasso
# (la penalización afecta a todos los coeficientes por igual)
```

```
# ── Ridge con búsqueda automática de alpha ──────────────────────────
alphas = np.logspace(-3, 3, 100)  # De 0.001 a 1000

ridge_cv = Pipeline([
    ('scaler', StandardScaler()),
    ('model',  RidgeCV(alphas=alphas, cv=5))
])
ridge_cv.fit(X_train, y_train)

best_alpha_ridge = ridge_cv.named_steps['model'].alpha_
r2_ridge = ridge_cv.score(X_test, y_test)
print(f'Ridge — alpha óptimo: {best_alpha_ridge:.4f}   R²: {r2_ridge:.4f}')

# ── Lasso con búsqueda automática de alpha ──────────────────────────
lasso_cv = Pipeline([
    ('scaler', StandardScaler()),
    ('model',  LassoCV(alphas=alphas, cv=5, max_iter=5000))
])
lasso_cv.fit(X_train, y_train)

best_alpha_lasso = lasso_cv.named_steps['model'].alpha_
r2_lasso = lasso_cv.score(X_test, y_test)
coefs_lasso = lasso_cv.named_steps['model'].coef_
n_zero = np.sum(coefs_lasso == 0)

print(f'Lasso — alpha óptimo: {best_alpha_lasso:.4f}   R²: {r2_lasso:.4f}')
print(f'Lasso eliminó {n_zero} de {len(coefs_lasso)} variables (coef = 0)')
```

3.4 — Visualización de coeficientes en función de alpha (Regularization Path)

```
from sklearn.linear_model import lasso_path, ridge_regression

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Lasso path
X_scaled = StandardScaler().fit_transform(X_train)
alphas_lasso, coefs_lasso_path, _ = lasso_path(X_scaled, y_train, alphas=alphas)

axes[0].plot(alphas_lasso, coefs_lasso_path.T)
axes[0].set_xscale('log')
axes[0].set_xlabel('Alpha (log scale)')
axes[0].set_ylabel('Coeficientes')
axes[0].set_title('Lasso Path  (coeficientes → 0 con alpha alto)')
axes[0].axhline(y=0, color='black', linestyle='--', lw=0.8)

# Ridge path (manual)
ridge_coefs = []
for a in alphas:
    coef = ridge_regression(X_scaled, y_train, alpha=a)
    ridge_coefs.append(coef)

axes[1].plot(alphas, ridge_coefs)
axes[1].set_xscale('log')
axes[1].set_xlabel('Alpha (log scale)')
axes[1].set_ylabel('Coeficientes')
axes[1].set_title('Ridge Path  (coeficientes → 0 pero nunca exactamente)')
```

```
axes[1].axhline(y=0, color='black', linestyle='--', lw=0.8)

plt.tight_layout()
plt.show()
```

3.5 — ¿Cuándo usar cada uno?

Situación	Recomendación
Todas las variables son teóricamente relevantes	Ridge
Se sospecha que muchas variables son ruido	Lasso
Variables fuertemente correlacionadas entre sí	Ridge (Lasso es inestable)
Se necesita un modelo interpretable y sparse	Lasso (actúa como selector)
No se sabe cuál usar	Elastic Net (combina L1 + L2)

Elastic Net: Combina ambas penalizaciones: $\alpha \cdot L1 + (1-\alpha) \cdot L2$. Disponible en sklearn como ElasticNet y ElasticNetCV. Es útil cuando hay grupos de variables correlacionadas y se desea selección + estabilidad.